

PART IV — FROM RAG TO AGENTIC RAG

Chapter 19B

Porting the Agent to a LangGraph State Machine

“Porting an agent to a graph is not a rewrite of its logic — it is a rewrite of everything the logic used to assume about being alone in one function.”

Chapter 19B — Porting the Agent to a LangGraph State Machine

Every structural choice in this chapter answers a question the single-file version never had to ask. Where does one phase's logic live when it becomes its own file? What happens to a shared helper function when two different node files both need it? How does a two-track parallel pipeline — the concrete trigger from

19B.1 The migration plan — from a loop to a node-per-phase graph

The plan was conservative by design, for the same reason Chapter 19.1's aborted first attempt failed: ``app/agent_query.py`` kept running, untouched, for the entire duration of the port. ``app_workflow/`` was built as a second package, importing nothing from ``app/`` and exporting nothing back to it, so a defect in the new graph could never take down the proven one. Every architecture decision record for this migration ends the same way — "``app/`` unaffected" — not as boilerplate, but as a standing constraint the team checked on every single change.

The plan itself reduces to one sentence: give every phase Chapter 18 hand-coded its own file, give every file exactly one job, and let ``graph.py`` be the only place that knows how the files connect. What used to be a phase variable and an ``if`` chain becomes a directory listing.

19B.2 One module per node

``app_workflow/nodes/`` holds sixteen files, and their names alone describe the pipeline without opening one.

Module	Responsibility
<code>`user_input.py`</code>	Entry point — parses commands, loads feedback context
<code>`query_variants.py`</code>	Generates and pre-filters query rephrasings
<code>`retrieve.py`</code>	One parallel branch per query variant
<code>`post_retrieve.py`</code>	Fan-in filter after all retrieve branches land
<code>`validate_retrieval.py`</code>	Per-track relevance judging
<code>`dedup_merge.py`</code>	Per-track near-duplicate chunk merging
<code>`nac.py`</code> / <code>`dc.py`</code> / <code>`lbc.py`</code>	The three compression stages (Chapter 22B)
<code>`combine_tracks.py`</code>	The fan-in barrier joining both compression tracks
<code>`generate_draft.py`</code> / <code>`generate_answer.py`</code>	The two-stage answer pipeline (Chapter 20.9)
<code>`check_answer_quality.py`</code>	The quality gate between them
<code>`no_context_answer.py`</code>	The canned-answer short-circuit
<code>`commands.py`</code>	<code>`bad`</code> / <code>`stats`</code> / <code>`learn`</code> / <code>`exit`</code> handlers
<code>`auto_distillation.py`</code>	Self-learning trigger after a good answer

Every module exports one function with the same shape: accept `GraphState`, return a partial update. There is no other contract to learn once that pattern is recognized once.

19B.3 The graph.py module — assembling nodes, edges, and entry points

`graph.py` does exactly one thing: register every node, declare every edge, and compile. It contains no business logic of its own — reading it top to bottom is reading the entire pipeline's shape, which is the payoff for having moved everything else out.

```
def build_graph():
    graph = StateGraph(GraphState)
    graph.add_node("retrieve", retrieve)
    graph.add_node("combine_tracks", combine_tracks, defer=True)
    ...
    graph.add_edge(START, "user_input")
    graph.add_conditional_edges("user_input", route_user_input, {...})
    ...
    return graph.compile()
```

Figure 19B.1 shows the correspondence directly: every phase Chapter 18 tracked with one `phase` variable now has a named file, and the state machine's transitions — the `if phase == "COMPRESS"` blocks — are now `graph.py`'s edge declarations.

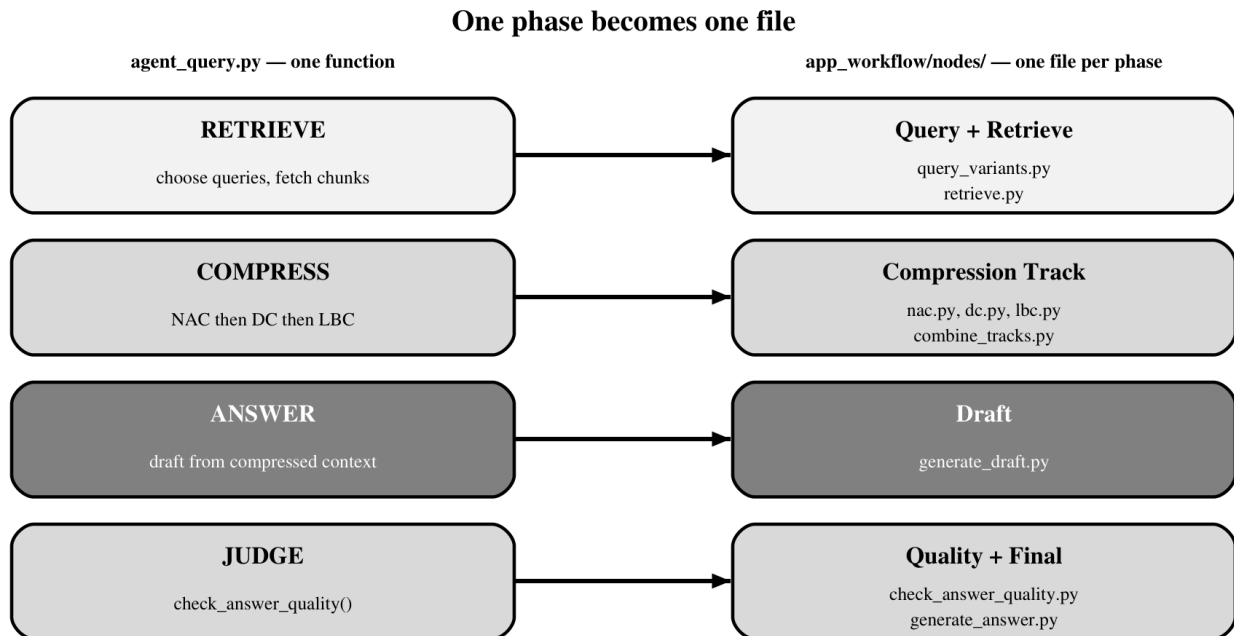


Figure 19B.1 — Chapter 18's four phases become fourteen node files, wired together in one place.

19B.4 The state.py schema

One file, `state.py`, declares every field any node anywhere in the graph may read or write — the complete `GraphState` Chapter 19.3 introduced in the abstract, now carrying this pipeline's actual shape: two raw retrieval tracks accumulated by reducer, two validated tracks, two dedup-merged tracks, a full document compression chain (NAC, DC, LBC) running parallel to a shorter learned-QA chain (DC, LBC only — no neighbor-merging step, because distilled Q&A chunks have no sequence to merge), and a combined output only `combine_tracks` is allowed to write.

Reading `state.py` end to end is the fastest way to understand what this graph actually does — faster than reading any single node, because every node's contract is legible from the fields it touches.

19B.5 The routes.py module

Every conditional-edge routing function in the entire graph lives in one file, imported by `graph.py` and nowhere else. Nothing about this is enforced by the language — it is a discipline the project chose, for the same reason `state.py` centralizes state: a reader who wants to know "what can happen after `combine_tracks`" should be able to find the answer in one place, not by tracing into node implementations.

```
def route_after_combine(state: GraphState) -> str:
    if not state.get("compressed_docs"):
        if state.get("retry_count", 0) >= LLM_RESPONSE_RETRY_LIMIT:
            return "no_context_answer"
        return "retry"
    return "generate_draft" if
get_switches(state)["ENABLE_ANSWER_DRAFT_CREATION"] else "generate_answer"
```

Nine routing functions live here, and every one of them reads state and returns a string — nothing more. `fan_out_retrievals` is the exception worth noting early: it returns a list of `Send` objects rather than a single string, because fanning out is a different shape of decision than choosing one path (Section 19B.8).

19B.6 The services/ layer

`app_workflow/services/` holds every piece of infrastructure a node might need but that has no idea a graph exists: `llm_caller.py` and `llm_setup.py` (Chapter 13B in full), `logger_config.py`, `prompts.py`, `embedding_manager.py`, `retriever.py`, `validators.py`, and `fix_llm_output.py` (Chapter 20B). None of these files import `GraphState`, `StateGraph`, or anything else from `LangGraph` — they are the same kind of reusable, framework-agnostic code Part III built, simply relocated under a package a graph now happens to sit on top of.

`services/services.py` is the one file in this layer worth naming specifically: a small module holding shared singleton instances — the live `retriever`, `feedback_store`, `self_learner`, and `learned_collection` objects — constructed once and imported by whichever node needs them. `commands.py`'s `cmd_bad` reaches into it directly: `services.last_variants_with_chunks`, the same variant-tracking data Chapter 18.6 threaded through `run_agent`'s return tuple, now living as shared state one module away instead of a value passed down a call stack.

19B.7 The `user_input_node` and `route_user_input`

The graph's entry point does everything `agent_query.py`'s command handling used to do inline, before a single retrieval happens: recognize `'bad'`, `'stats'`, `'learn'`, and `'exit'` as commands rather than questions, and — for an actual question — load blocked variants and prior thumbdowns from the feedback store so they can steer retrieval exactly as Chapter 18's `'_build_system_prompt'` did.

```
def user_input_node(state: GraphState) -> dict:
    raw_lower = state["user_input"].strip().lower()
    if raw_lower in {"exit", "quit"}:
        return {"command": "exit"}
    ...
    return {
        "command": "",
        "blocked_variants": blocked_variants,
        "prior_thumbdowns": prior_thumbdowns,
    }
```

`'route_user_input'` then reads `'command'` and sends the run to a dedicated node per command, or on to `'generate_query_variants'` for anything that was not a command at all. Command handling and question handling are structurally the same kind of decision — a conditional edge — rather than the special-cased early-return checks a hand-written loop tends to accumulate.

19B.8 Fan-out — two independent retrievals from a single START

This is the concrete capability Chapter 19.1 identified as the actual trigger for the migration: multiple query variants, each retrieving independently, running as `'max(variants)'` instead of `'sum(variants)'`. `'fan_out_retrievals'` returns one `'Send'` per variant, each carrying its own copy of state with `'query'` overridden — LangGraph schedules every `'Send'` as an independent parallel task.

```
def fan_out_retrievals(state: GraphState):
    return [
        Send("retrieve", {**state, "query": variant})
        for variant in state["query_variants"]
    ]
```

`'run_agent'`'s hand-written loop never attempted this — it could not, without threading or async machinery a single Python function has no clean way to express. A conditional edge that returns a list of `'Send'` objects instead of one node name is the entire mechanism; nothing about `'retrieve.py'` itself needed to change to become parallelizable.

19B.9 Fan-in via reducer-typed list fields

Every parallel `retrieve` branch writes to the same two state fields, and `state.py`'s reducers are what make that safe: `retrieved_document_chunks` and `retrieved_learned_qa_chunks` are both `Annotated[list[dict], operator.add]`, so `LangGraph` concatenates every branch's contribution rather than letting the last branch to finish silently overwrite the others.

```
retrieved_document_chunks: Annotated[list[dict], operator.add]
retrieved_learned_qa_chunks: Annotated[list[dict], operator.add]
variants_with_chunks: Annotated[list[dict], operator.add]
newly_failed_variants: Annotated[list[str], operator.add]
```

`post_retrieval_filter` sits immediately after `retrieve` in the graph and runs once, implicitly waiting for every fanned-out branch to finish before it fires — an ordinary, non-deferred fan-in, correct here because every branch writes into reducer-typed fields rather than fields a barrier would need to protect. Section 19B.10 is where that stops being sufficient.

19B.10 The `combine_tracks` node

The document and learned-QA compression tracks run at different depths — three compression stages against two — and land in different graph supersteps even though both start from the same fan-out point. `combine_tracks` is registered with `defer=True` specifically because a plain multi-predecessor node would fire once per arriving edge instead of once for both.

The node versus service boundary

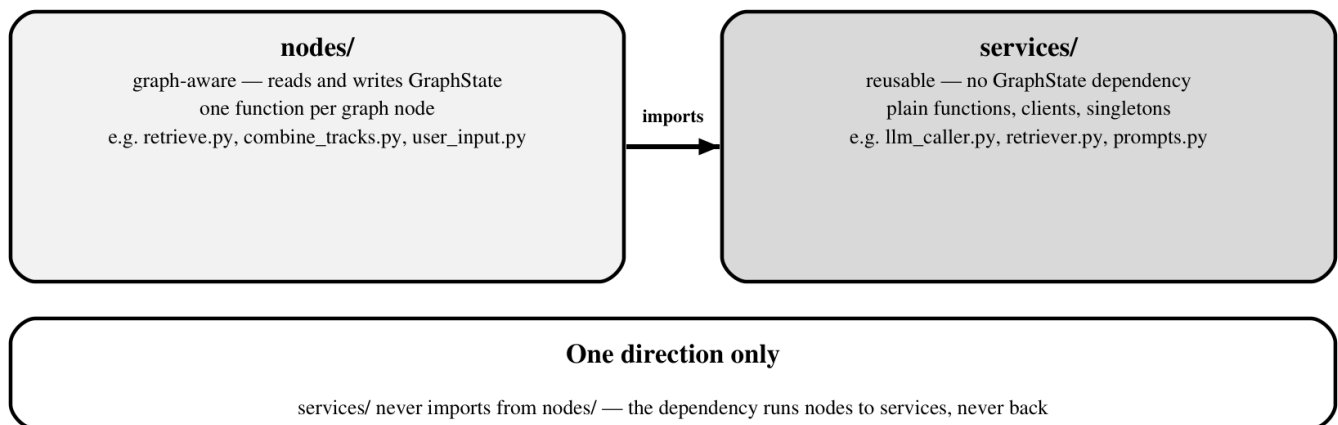


Figure 19B.2 — Nodes depend on services; services never depend on nodes.

(Figure 19B.2 is introduced here to set up Section 19B.13 — hold that thought for two sections.) `combine_tracks` itself is almost aggressively simple once the barrier guarantees both tracks are present: concatenate learned-QA chunks first, documents second, matching the same precedence ordering `agent_query.py`'s `format_precedence_context_for_llm` already established in Chapter 18. The barrier is where the real engineering is; the join itself is one list concatenation.

19B.11 The draft, quality-check, and final-answer routing

Three routing functions reproduce Chapter 18's JUDGE-phase logic exactly, as three separate, individually testable functions instead of branches inside one large conditional block.

Routing function	Mirrors	Decision
<code>`route_after_combine`</code>	COMPRESS → ANSWER transition	Empty result → retry or give up; else proceed to draft
<code>`route_after_generate_draft`</code>	Draft-creation feature flag	Skip straight to <code>`generate_answer`</code> if drafting is disabled
<code>`route_after_quality_check`</code>	JUDGE phase's <code>budget_ok</code> logic	OK → answer; INSUFFICIENT + budget → retry; else best-effort

``route_after_quality_check``'s budget check is worth comparing line for line against Chapter 18.5's inline version — the logic is unchanged, only its address changed, from a nested ``if`` inside ``run_agent`` to a standalone function ``graph.py`` wires in by name.

19B.12 Maintaining both pipelines side-by-side

``app/`` and ``app_workflow/`` ran in parallel for weeks, not days — long enough for ``app_workflow/`` to accumulate its own bugs, its own tuning passes, and its own provider migration (Chapter 13B) entirely independent of the proven loop sitting next to it. Every architecture decision touching the new graph states its blast radius explicitly: "``app/`` unaffected," repeated so consistently across the record that it reads less like a note and more like a checklist item nobody was willing to skip.

The discipline is what made the migration safe to attempt at all. A team free to break the working version while building its replacement has no fallback when the replacement's own bugs surface — and Chapter 19B.14's circular import was found precisely because someone could still run the old pipeline to confirm the new one, not the underlying logic, was what had broken.

19B.13 The node-vs-service architectural boundary

Figure 19B.2's rule is simple to state and easy to violate by accident: a ``nodes/`` file may import from ``services/``, never the reverse, and a ``services/`` file should never need to know ``GraphState`` exists at all. ``llm_caller.py`` living in ``services/`` and not ``nodes/`` is not a naming preference — it is the same file, doing the same job, that Chapter 13B described for the pre-graph pipeline, proof that centralizing LLM invocation was never actually coupled to which orchestration layer sits above it.

The test is practical: if a function needs ``GraphState`` to make sense — it reads or writes specific graph fields — it belongs in ``nodes/``. If it would work identically whether the caller were a graph node, a CLI script, or a unit test, it belongs in ``services/``. A ``services/`` file that starts needing ``state.get(...)`` is a sign the boundary has already been crossed, quietly, and is worth catching before it spreads.

19B.14 Circular imports across nodes/

``app_workflow/`` sits on ``sys.path`` two different ways depending on entry point — ``main.py`` imports it as the bare ``nodes`` package; ``api.py``, run via uvicorn from inside ``app_workflow/``, can end up importing the same files under the fully-qualified ``app_workflow.nodes`` identity instead. Python does not deduplicate these — two different import paths to the same file produce two different module objects, each with its own copy of every name defined in it.

Every file in ``nodes/`` used the bare ``nodes.X`` form to reference a sibling module, matching ``nodes/__init__.py`` itself — except one line in ``generate_answer.py``, which imported ``check_answer_quality`` via the absolute ``app_workflow.nodes.check_answer_quality`` path instead. Because ``__init__.py`` imports ``generate_answer`` before ``check_answer_quality`` in its own declared order, that one absolute import re-entered the still-initializing ``nodes`` package under its *other* identity partway through, and failed to find ``generate_answer`` defined yet — ``ImportError: cannot import name 'generate_answer' from partially initialized module 'nodes.generate_answer'``.

Common pitfall — One inconsistent import style, two identities for the same package

The bug reproduced reliably from ``main.py`` and did not reproduce from ``api.py`` in the same session — not because the code was only sometimes wrong, but because only one entry point happened to trigger the exact import order that exposed it. A defect that depends on which door you walked in through is still a defect; test every real entry point, not just the one you use during development. The fix was one line — match the bare ``nodes.X`` style every other file in the package already used — but finding it required recognizing that ``sys.path`` position, not the file's own code, was what had two different opinions about what package this file belonged to.

Detecting this class of bug before it reaches a user is mostly a matter of consistency, not cleverness: pick one import style for intra-package references — bare, relative, or fully-qualified — and enforce it everywhere, because the moment two styles coexist, Python's import system is free to load the same file twice under two names, and only one of them will ever be the one already halfway through initializing.

``app_workflow/`` is now a complete, parallel agent — every phase Chapter 18 hand-coded, reproduced as a graph, with a genuine parallel retrieval fan-out the original loop structurally could not express. What has not yet changed is how good any of its answers are.